Obstacle Avoiding Real-Time Trajectory Generation of Omnidirectional Vehicles

Ji-wung Choi, Renwick E. Curry and Gabriel Hugh Elkaim

Abstract— **In this paper, a computationally effective trajectory generation algorithm of omnidirectional mobile robots is proposed. The algorithm plans a reference path based on Bézier curves, which meets obstacle avoidance. Then the algorithm solves the problem of motion planning for the robot to track the path in short travel time while satisfying dynamic constraint. Accelerations of the robot are computed such that they satisfy the time optimal condition for each sample time. The numerical simulation demonstrates the improvement of trajectory generation in terms of travel time, satisfaction of dynamic constraint and smooth motion control compared to a previous research.**

I. INTRODUCTION

Many researchers have worked on motion planning of the vehicles. The form of the vehicle includes car-like, differential drive, omni-directional, and other models. Balkcom [4] developed the time optimal trajectories for the bounded velocity model of differential drive robots. Jung [5] and Moore [6] dealt with omnidirectional vehicles; the control strategy employed by these papers consist of building a geometric path and tracking path by using feedback control. Huang [7] proposed a approach to vision-guided local navigation for nonholonomic robots, based upon a model of human navigation. The approach uses the relative headings to the goal and to obstacles, the distance to the goal, and the angular width of obstacles, to compute a potential field over the robot heading. This potential field controls the angular acceleration of the robot, steering it towards the goal and away from obstacles. Hamner [8] maneuvered an outdoor mobile robot that learns to avoid collisions by observing a human driver operate a vehicle equipped with sensors that continuously produce a map of the local environment. The paper describes implementation of steering control that models human behavior in trying to avoid obstacles while trying to follow a desired path. Hwang [9] developed the trajectory tracking and obstacle avoidance of a car-like mobile robot within an intelligent space via mixed H_2/H_∞ decentralized control. Two CCD cameras are used to realize the pose of the robot and the position of the obstacle. Based on the authority of these cameras, a reference command for the proposed controller of the robot is planned.

This paper especially focuses on two papers: Kalmar-Nagy [2] and Sahraei [1]. Kalmar-Nagy [2] has proposed minimum time trajectory generation algorithm for omnidirectional vehicles, which meets dynamic constraints, but no obstacles are considered. A near-optimal control strategy is shown to be piecewise constant (bang-bang type) in the paper. Sahraei [1] has presented a motion planning algorithm for omnidirectional vehicles, based on the result of [2]. The paper has claimed that the algorithm satisfies obstacle avoidance as well as time optimality given in discrete time system.

However, this paper ends up with that Sahraei's algorithm is problematic. To resolve the problems, a new motion planning algorithm for omnidirectional vehicles is proposed, which also satisfies obstacle avoidance and dynamic constraint in discrete time system. The numerical simulation provided in this paper demonstrates better solution to the problem of motion planning by the proposed algorithm than Sahraei's. This paper is organized as follows. Section III describes dynamic constraints of the robots based on the result of [2]. In section IV, Sahraei's algorithm [1] is introduced. Section V proposes the new algorithm. Finally, a numerical simulation is presented in Section VI.

II. BACKGROUND

A. Bézier Curve

Bézier Curves were invented in 1962 by the French engineer Pierre Bézier for designing automobile bodies. Today Bézier Curves are widely used in computer graphics and animation. A Bézier Curve of degree n can be represented as

$$P(\lambda) = \sum_{i=0}^{n} B_i^n(\lambda) P_i, \quad \lambda \in [0,1] \tag{1}$$

$$B_i^n(\lambda) = \binom{n}{i} (1-\lambda)^{n-i} \lambda^i, \quad i \in \{0,1,\ldots,n\} \tag{2}$$

Bézier Curves have useful properties for path planning:

- They always passes through P_0 and P_n.
- They are always tangent to the lines connecting $P_0 \to P_1$ and $P_n \to P_{n-1}$ at P_0 and P_n respectively.
- They always lie within the convex hull consisting of their control points.

J. Choi is a Ph.D. student at Autonomous Systems Lab, Computer Engineering Department in University of California, Santa Cruz, 95064, USA. He received BSE from Yonsei University in South Korea. jwchoi@soe.ucsc.edu

R. Curry is an president of Applied Aeronautical Systems Inc. He received an AB degree in Physics from Middlebury College, and the S.B., S.M., Engineer of A.A., and Ph.D. degrees in Aeronautics and Astronautics from the Massachusetts Institute of Technology (MIT). rcurry@ucsc.edu

G. Elkaim is an assistant professor at Autonomous Systems Lab, Computer Engineering Department in University of California, Santa Cruz, 95064, USA. He received BSE from Princeton University, and MSE and Ph.D. from Stanford. elkaim@soe.ucsc.edu

III. Dynamic Constraints of the Omnidirectional Vehicle

Fig. 1 shows the bottom view of an omnidirectional vehicle that consists of three wheels. This type of vehicle is able to move in any direction and spin as it moves. Kalmar-Nagy described an accurate model that relates the amount of torque available for acceleration to the speed of the three wheeled omnidirectional vehicle [1]. This section is based on the results of [2].

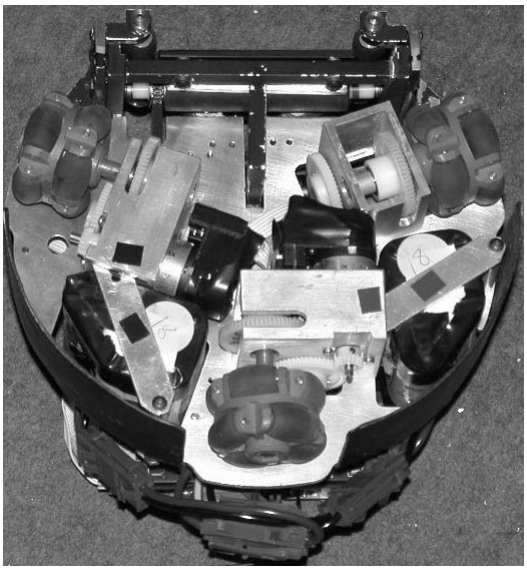


Fig. 1. Bottom view of the omnidirectional vehicle [2]

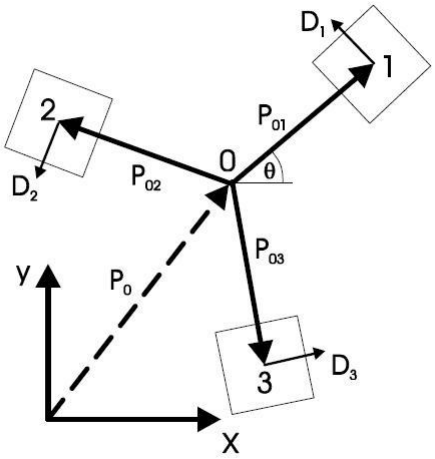


Fig. 2. Geometry of the omnidirectional vehicle [2]

It is shown that the drive velocities are defined as linear functions of the velocity and the angular velocity of the robot:

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} -\sin\theta & \cos\theta & L \\ -\sin(\frac{\pi}{3}-\theta) & -\cos(\frac{\pi}{3}-\theta) & L \\ -\sin(\frac{\pi}{3}+\theta) & -\cos(\frac{\pi}{3}+\theta) & L \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}, \quad (3)$$

where L is the distance of the drive units from the center of mass of the robot, v_i is the individual wheel velocities, θ is the angle of counterclockwise rotation (See Fig. 2). The new time and length scales are introduced

$$T = \frac{2m}{3\beta}, \quad \Psi = \frac{4\alpha m U_{max}}{9\beta^2}, \quad (4)$$

to normalize x, y, and t to the nondimensional variables

$$\bar{x} = \frac{x}{\Psi}, \quad \bar{y} = \frac{y}{\Psi}, \quad \bar{t} = \frac{t}{T}. \quad (5)$$

The constants α and β are determined by the motor character. U_{max} is the maximum value of the voltage applied to the motor. m is the mass of the robot. Then the constraint of the robot (after dropping the bars) becomes

$$q_x^2(t) + q_y^2(t) \leq 1, \quad (6)$$

where the two components of control $q_x(t)$ and $q_y(t)$ are

$$q_x(t) = \ddot{x} + \dot{x}, \quad (7)$$

$$q_y(t) = \ddot{y} + \dot{y}. \quad (8)$$

It has been shown that the optimal control strategy is achieved when

$$q_x^2(t) + q_y^2(t) = 1, \quad t \in [0, t_f], \quad (9)$$

where t_f is the final time. Kalmar-Nagy [2] solves the problem of time optimal motion trajectory by ensuring the equality, but no obstacles are considered [1].

IV. Sahraei's Algorithm

Sahraei [1] proposed a trajectory generation algorithm based on the result of [2]. The algorithm is differentiated from Kalmar-Nagy's algorithm by two properties: real-time trajectory generation and obstacle avoidance. The first step is to construct the Voronoi diagram to find a sketch path that keeps away from obstacles. Start and target points, s and t are added to this graph with corresponding edges which connect these two points to their cells vertices. After constructing the graph Dijkstras shortest path algorithm is run. The resulting path is the shortest path whose edges are in the Voronoi diagram. Two Bézier curves are used to find a smooth path near the resulting path with regards to initial and final conditions. Let $p_0, p_1, \ldots, p_n$ are vertices of the shortest path and p_0, and p_n are s and t respectively. The first Bézier curve, $P_a(\lambda)$ for $\lambda \in [0,1]$, is constructed by p_0, q, r, and p_1, where control points q and r are introduced to satisfy slope of initial velocity constraint and continuity of curve and its slope in p_1. The second Bézier curve $P_b(\lambda)$ is constructed by $p_1, \ldots, p_n$. Following equations describe boundary conditions:

$$\frac{\dot{P}_a(0)}{|\dot{P}_a(0)|} = \frac{v_0}{|v_0|}, \quad (10)$$

$$\frac{\dot{P}_a(1)}{|\dot{P}_a(1)|} = \frac{\dot{P}_b(0)}{|\dot{P}_b(0)|}. \quad (11)$$

Fig. 3 shows an example of the paths.

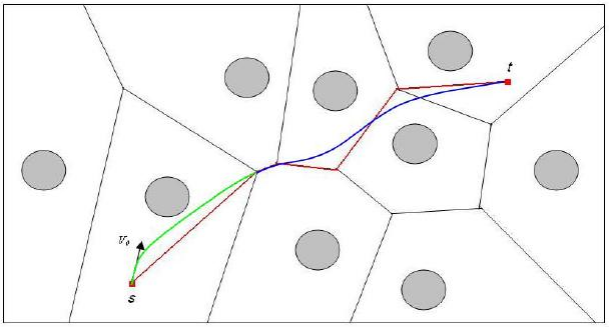


Fig. 3. A smooth path resulted from two Bezier curves. The first Bezier curve is illustrated in green and the second one is shown in blue [1].

Finally Sahraei assigned velocity magnitude to each point on the generated curve $P(\lambda) = \big(X(\lambda), Y(\lambda)\big)$. To implement this, the paper tried to find a function $\alpha : t \in \{0, h, 2h, \ldots\} \rightarrow \lambda \in [0,1]$ such that $X(\alpha(t))$ and $Y(\alpha(t))$ satisfy the following dynamic constraint

$$(\dot{X} + \ddot{X})^2 + (\dot{Y} + \ddot{Y})^2 \leq 1, \tag{12}$$

where $h \in \mathbb{R}^+$ is the sample time interval. Note that the variables X, Y, and t for (12) are normalized values by (4). For the sake of optimality $\alpha(t)$ was calculated such that the left side of the inequality constraint (12) approaches 1. To find $\lambda_n \triangleq \alpha(nh)$ for all n's Sahraei used derivative approximations to define the function f:

$$\begin{aligned} f(\lambda) = \quad & \left(\tfrac{X(\lambda)-X(\lambda_{n-2})}{2h} + \tfrac{X(\lambda)+X(\lambda_{n-2})-2X(\lambda_{n-1})}{h^2}\right)^2 \\ & + \left(\tfrac{Y(\lambda)-Y(\lambda_{n-2})}{2h} + \tfrac{Y(\lambda)+Y(\lambda_{n-2})-2Y(\lambda_{n-1})}{h^2}\right)^2 - 1 + \varepsilon. \end{aligned} \tag{13}$$

λ_n is calculated by solving the equation

$$f(\lambda) = 0 \tag{14}$$

based on λ_{n-1} and λ_{n-2} by Newton's method. $\varepsilon \in \mathbb{R}^+$ in (13) guarantees that the results of Newton's method make the left side of inequality (12) less than but close to 1. Since λ_n relies on two previous values of λ, at the first step λ_0 and λ_1 should be calculated. It is straightforward that $\lambda_0 = 0$. To calculate λ_1, a hypothetical λ_{-1} is introduced to approximate the position of the robot before initial time, $X(\lambda_{-1})$ defined by

$$\frac{X(\lambda_0) - X(\lambda_{-1})}{h} \approx v_{x_0}, \tag{15}$$

and so forth for Y.

V. Proposed Algorithm

This section proposes a new algorithm for obstacle avoiding real-time trajectory generation of omnidirectional vehicles. To describe this method, let $a_n = (a_{x_n}, a_{y_n})$, $v_n = (v_{x_n}, v_{y_n})$, $z_n = (x_n, y_n)$ denote the acceleration, velocity, and position of the vehicle, respectively, at sample time $t = nh$. All of them are normalized values by (4).

The new algorithm uses Voronoi's diagram and a Bézier curve to generate the reference trajectory as Sahraei did. A problem of Sahraei's algorithm is that it did not do any calculation to ensure that the Bézier curve misses obstacles. To resolve this problem, this paper tests if convex hull constructed by control points of the Bézier curve does not intersect obstacles. If there is any intersections between them, the control points should be replaced so that the intersections disappear.

The algorithm also deals with velocities and accelerations of the robot in discrete time system sampled by $t = \{0, h, 2h, \ldots\}$. However, it computes accelerations a_n as opposed to that Sahraei computes positions z_n in order to meet optimality condition (9). The set of all such accelerations, $\mathfrak{A}_n^p$ is represented as

$$\mathfrak{A}_n^p = \{(a_{x_n}, a_{y_n}) | \quad (a_{x_n} + v_{x_n})^2 + (a_{y_n} + v_{y_n})^2 = 1\}. \tag{16}$$

If v_n is given, (16) can be rewritten as

$$\mathfrak{A}_n^p = \{(-v_{x_n} + \cos\theta_n, -v_{y_n} + \sin\theta_n) | \quad \theta_n \in [0, 2\pi)\}. \tag{17}$$

The beauty of (17) is to guarantees satisfaction of (9) while Sahraei's algorithm only has the left side of the equation approach 1. It also simplifies the value of the accelerations to one variable θ_n.

In this algorithm, the robot is assumed to follow the constant acceleration equations of motion with a_n for time interval $t \in \big[nh, (n+1)h\big)$. Once a_n is determined, thus, the velocity and the position at next sample time is calculated by applying the constant acceleration motion equations and using a_n:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + h\mathbf{a}_n, \quad \mathbf{a}_n \in \mathfrak{A}_n^p, \tag{18}$$

$$\mathbf{z}_{n+1} = \mathbf{z}_n + h\mathbf{v}_n + \frac{h^2}{2}\mathbf{a}_n, \quad \mathbf{a}_n \in \mathfrak{A}_n^p. \tag{19}$$

In the problems that we consider, v_0 and z_0 are initially given. So a_0 is solely determined by selecting θ_0 in (17). Once a_0 is determined, v_1 and z_1 are obtained by applying (18) and (19) and using a_0, and so on. Thus we only need to find θ_n for all n's in order to fulfill motion planning of the robot, represented by a set of z_n. We also can generalize that v_n and z_n are give when we calculate θ_n at $t = nh$.

Equation (19) can be rewritten as the sum of two vectors:

$$\mathbf{z}_{n+1} = \mathbf{c}_{n+1} + \mathbf{r}_{n+1}, \tag{20}$$

where

$$\mathbf{c}_{n+1} = \begin{bmatrix} x_n \\ y_n \end{bmatrix} + (h - \frac{h^2}{2}) \begin{bmatrix} v_{x_n} \\ v_{y_n} \end{bmatrix}, \tag{21}$$

$$\mathbf{r}_{n+1} = \frac{h^2}{2} \begin{bmatrix} \cos\theta_n \\ \sin\theta_n \end{bmatrix}. \tag{22}$$

Since v_n and z_n are given at $t = nh$, $\mathbf{c}_{n+1}$ is deterministic. $\mathbf{r}_{n+1}$ relies on θ_n. So the set of all feasible z_{n+1} corresponding to $\mathfrak{A}_n^p$, $\mathfrak{Z}_{n+1}^p$ is given by

$$\mathfrak{Z}_{n+1}^p = \{\mathbf{z}_{n+1} | \quad |\mathbf{z}_{n+1} - \mathbf{c}_{n+1}| = \tfrac{h^2}{2}\}. \tag{23}$$

Geometrically, $\mathfrak{Z}_{n+1}^p$ can be interpreted as the circle that has center at $\mathbf{c}_{n+1}$ and radius of $\frac{h^2}{2}$ as shown in Fig. 4(a).

The perspective points at next sample time, z_1^p and z_2^p are the intersect points between the circle and the pre-generated Bézier curve $P(\lambda) = \big(X(\lambda), Y(\lambda)\big)$. We will select z_1^p, the one further down from current position as z_{n+1} for shorter travel time.

θ_n corresponding to z_1^p can be calculated in computationally efficient way. Firstly, we define the point on the Bézier curve, p which is the closest to c_{n+1} as shown in Fig. 4(a). To calculate p, we introduce the function f:

$$f(\lambda) = (X(\lambda) - c_{x_{n+1}})^2 + (Y(\lambda) - c_{y_{n+1}})^2. \tag{24}$$

Then p is given by

$$p = \big(X(\lambda^p), Y(\lambda^p)\big), \tag{25}$$

where λ^p is the solution to the equation $f(\lambda) = 0$, which is calculated by applying Newton's method. In Fig. 4(a), the portion of the Bézier curve inside of the circle can be considered to be line segment of which slope is equal to the slope of tangent at p, given that time interval h is small enough. So we can approximate z_1^p as the intersect point between the circle and the tangent line at p. Let ϕ denote the slope of the tangent line:

$$\phi = \tan^{-1}\big(\frac{\dot{Y}(\lambda^p)}{\dot{X}(\lambda^p)}\big). \tag{26}$$

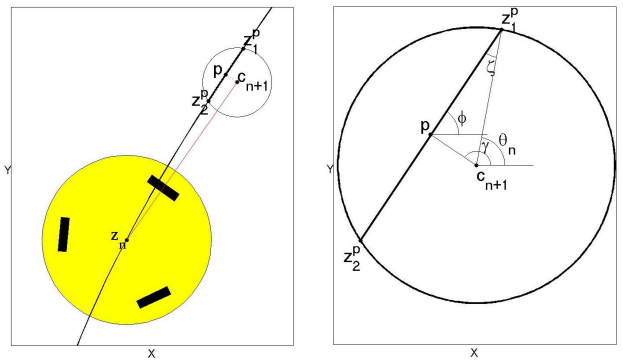
Looking at the geometry of p, z_1^p, and c_{n+1} in Fig. 4(b), θ_n is given by

$$\theta_n = \phi + \zeta, \tag{27}$$

where ζ can be calculated by applying law of sines for $\triangle z_1^p p c_{n+1}$:

$$\zeta = \sin^{-1}\big(\frac{2|p - c_{n+1}|}{h^2}\sin(\pi - \gamma + \phi)\big). \tag{28}$$

Where γ is the signed angle of direction of the vector $\overrightarrow{c_{n+1}p}$.



(a) $\mathfrak{Z}_{n+1}^p$ on a reference trajectory. (b) The enlarged $\mathfrak{Z}_{n+1}^p$.

Fig. 4. Geometry of $\mathfrak{Z}_{n+1}^p$ from $\mathbf{z}_n$ on a reference trajectory. p is defined by the point on the reference trajectory, which is closet to $\mathbf{c}_{n+1}$

VI. Numerical Simulations

Simulations provided in this section demonstrates improvement of trajectory generation by the proposed algorithm in terms of travel time, satisfaction of dynamic constraint and smooth motion control compared to Sahraei's algorithm. Fig. 5 shows the course used for the simulation. Red circles indicate obstacles. The initial position and the velocity are given by:

$$z_0 = (1.75, 0.54) \quad [m] \tag{29}$$
$$v_0 = (0, 0) \quad [m/s] \tag{30}$$

The final position is given by

$$z_f = (6.85, 3.28) \quad [m] \tag{31}$$

The sample time interval h is given by

$$h = 0.0033[s]. \tag{32}$$

Characteristic variables are given by

$$\alpha = 1[N/V], \quad \beta = 1[kg/s], \quad m = 1[kg], \quad Umax = 3[v] \tag{33}$$

The reference trajectory is constructed by a Bézier curve of

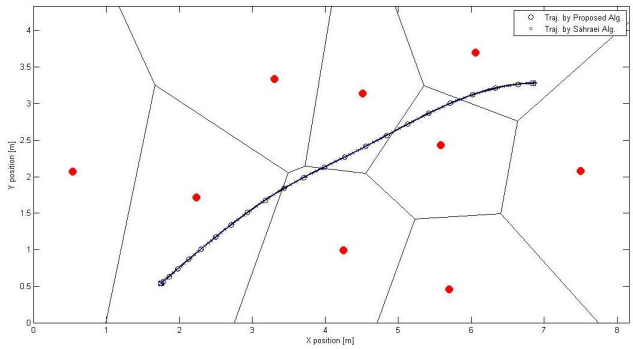
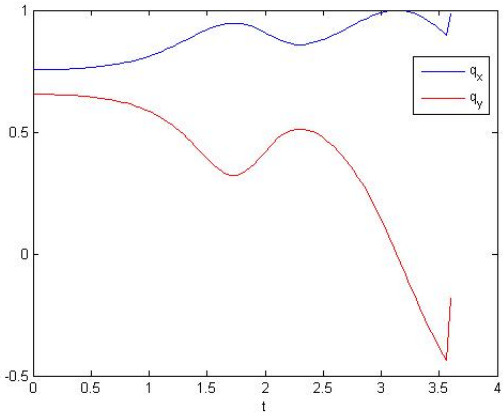


Fig. 5. The resulting trajectories by two different algorithms over the reference trajectory (bold black curve).

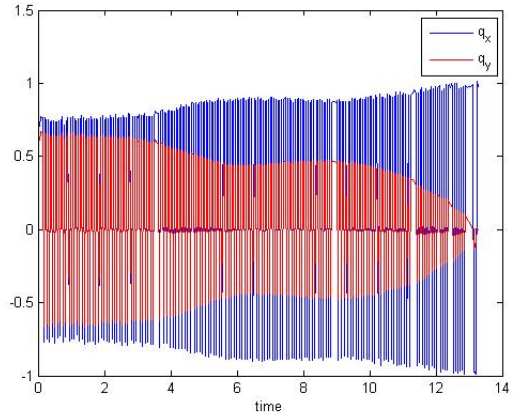
which control points are given by

$$p_0 = (1.75, 0.54), p_1 = (3.49, 2.05), p_2 = (3.72, 2.14),$$
$$p_3 = (4.55, 2.04), p_4 = (5.35, 3.24), p_5 = (6.85, 3.28) \tag{34}$$

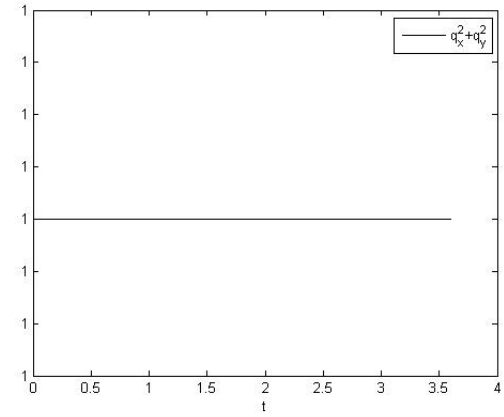
The simulation of Sahraei's algorithm has been done with the same parameters above and $\varepsilon = 0.01$. In Fig. 5, the generated trajectories by the proposed algorithm and Sahraei's algorithm are plotted. The simulation results of two algorithms are listed in table I. The resulting final time t_f by the proposed algorithm is quite shorter than the one by Sahraei's algorithm. Sahraei's algorithm leads to dynamic constraint (6) violation. We can see that $q_x^2 + q_y^2$ exceeds boundary condition 1 in Fig. 6(d). In other hand, $q_x^2 + q_y^2$ by the proposed algorithm is always 1 as shown in Fig. 6(c). In addition, the proposed algorithm generates quite smoother controls, q_x and q_y and velocities v_x and v_y than Sahraei's algorithm as shown in Fig. 6(a) and 6(e).



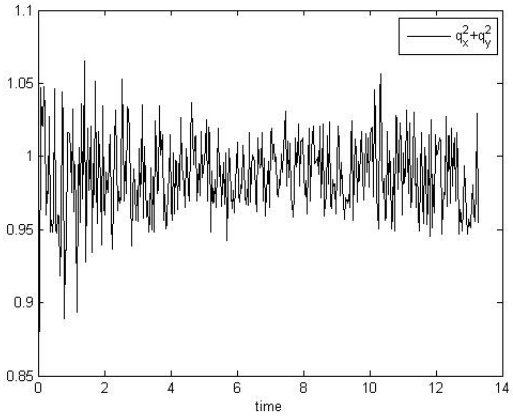
(a) q_x and q_y by the proposed algorithm.



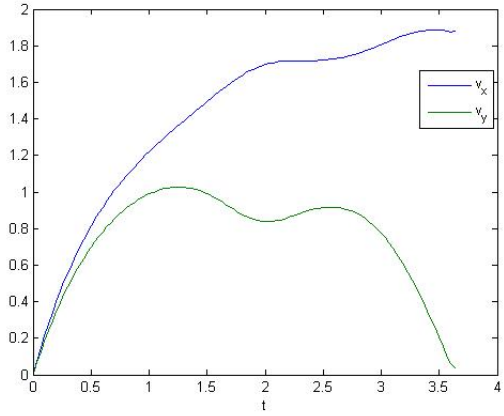
(b) q_x and q_y by Sahraei's algorithm.



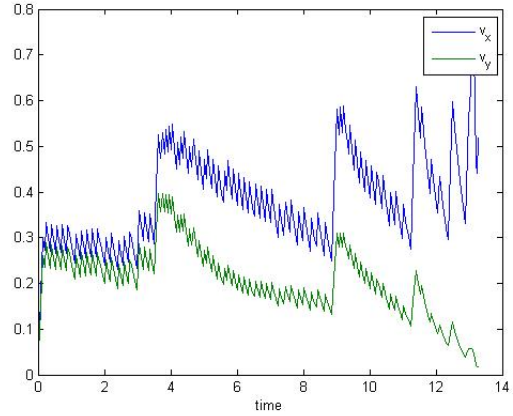
(c) $q_x^2 + q_y^2$ by the proposed algorithm.



(d) $q_x^2 + q_y^2$ by Sahraei's algorithm.



(e) v_x and v_y by the proposed algorithm.



(f) v_x and v_y by Sahraei's algorithm.

Fig. 6. The results obtained by two algorithm.

TABLE I

RESULTS OF THE SIMULATION

Algorithm	t_f $[s]$	Violation of (6) $[\%]$
Proposed Algorithm	3.6333	0
Sahraei's Algorithm	13.2667	31.91

VII. CONCLUSIONS

This paper proposes a collision-free real-time motion planning algorithm for omnidirectional mobile robot. It has been shown that motion of the robot is planned in computationally effective way in order to satisfy obstacle avoidance and generate short travel time trajectory by applying the proposed algorithm. Numerical simulations demonstrates improvement of the motion planning compared to Sahraei's algorithm.

REFERENCES

[1] A. Sahraei, M. T. Manzuri, M. R. Razvan, M. Tajfard and S. Khoshbakht, "Real-Time Trajectory Generation for Mobile Robots," *The 10th Congress of the Italian Association for Artificial Intelligence (AIIA 2007)* September 10-13, 2007 .

[2] Kalmar-Nagy T., D'Andrea R., Ganguly P., "Near-optimal dynamic trajectory generation and control of an omnidirectional vehicle," *Robotics and Autonomous Systems, Volume 46, Number 1,* 31 January 2004 , pp. 47-64(18), Elsevier.

[3] Athans, M. and Falb, P. L., "Optimal Control: An introduction to the theory and its applications," McGraw-Hill, New York, 1966.

[4] Balkcom, D. and Mason, M. Time Optimal, "Trajectories for Bounded Velocity Differential Drive Robots," *IEEE International Conference on Robotics and Automation (ICRA 00),* p. 2499 - 2504, 2000.

[5] Jung, M., Shim, H., Kim, H. and Kim, J., "The Miniature Omnidirectional Mobile Robot OmniKity-I (OK-I)," *International Conference on Robotics and Automation,* p. 2686-2691, 1999.

[6] Moore, K. L. and Flann, N. S., "Hierarchial Task Decomposition Approach to Path Planning and Control for an Omni-Directional Autonomous Mobile Robot," *International Symposium on Intelligent Control/Intelligent Systems and Semiotics,* p. 302-307, 1999.

[7] Huang, W. H., Fajen, B. R., Fink, J. R., Warren, W. H., "Visual navigation and obstacle avoidance using a steering potential function," *Robotics and Autonomous Systems,* vol. 54, Issue 4, p. 288-299, 28 April 2006.

[8] Hamner, B., Singh, S., Scherer, Se., "Learning Obstacle Avoidance Parameters from Operator Behavior," *Special Issue on Machine Learning Based Robotics in Unstructured Environments, Journal of Field Robotics,* vol. 23, 11/12, p. 1037-1058, December 2006.

[9] Hwang, C., Chang, L. "Trajectory Tracking and Obstacle Avoidance of Car-Like Mobile Robots in an Intelligent Space Using Mixed H_2/H_∞ Decentralized Control," *Mechatronics, IEEE/ASME Transactions on,* vol. 12, Issue 3, p. 345-352, June 2007